



MARMARA
ÜNİVERSİTESİ

T.C. MARMARA ÜNİVERSİTESİ TEKNOLOJİ FAKÜLTESİ

2025-2026 EĞİTİM ÖĞRETİM YILI BAHAR DÖNEMİ

MTH3018.1 – Bulut Mimarilerinde Test Mühendisliği Dönem Projesi Raporu

Ders Adı: Bulut Mimarilerinde Test Mühendisliği

Proje Adı: To-Do List Manager

Eğitmen Adı: Büşra Ayaksız

Öğrenci Adı Soyadı: Ahmet Gemici

Öğrenci Numarası: 170423041

1. Giriş

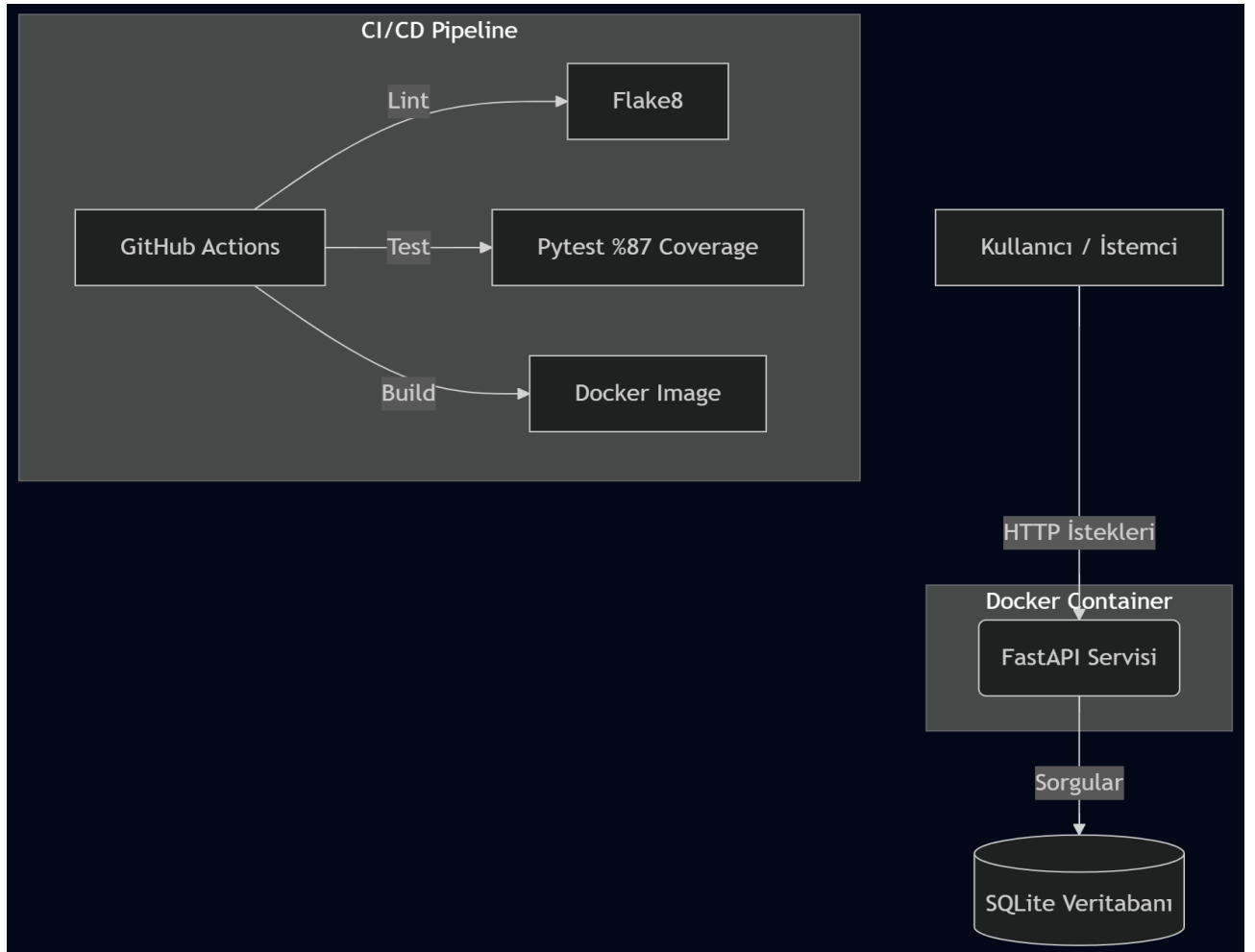
Günümüz yazılım geliştirme süreçlerinde, uygulamaların bulut ortamlarına güvenli ve kesintisiz bir şekilde aktarılabilmesi için uçtan uca (E2E) test boru hatlarının (pipeline) kurulması kritik bir öneme sahiptir. Bu projenin temel amacı, "Bulut Mimarilerinde Test Mühendisliği" dersi kapsamında öğrenilen modern test araçlarını, CI/CD süreçlerini ve konteyner mimarilerini pratik bir uygulama üzerinde birleştirmektir.

Bu amaç doğrultusunda mini-uygulama olarak bir "To-Do List Manager" (Görev Yöneticisi) mikroservisi seçilmiştir. Bu konunun seçilmesindeki temel motivasyon; domain mantığının (iş kurallarının) nispeten sade olması ve böylece asıl odak noktası olan altyapı, test stratejileri ve otomasyon süreçlerine (Docker, GitHub Actions, Pytest) çok daha fazla zaman ve emek ayrılabilmesidir.

2. Mimari

Projenin mimarisi, hafiflik ve yüksek performans prensiplerine dayanarak tasarlanmıştır.

- **Servis Katmanı:** Uygulamanın bel kemiğini, Python tabanlı modern ve asenkron destekli web framework'ü olan FastAPI oluşturmaktadır. FastAPI, hem hızlı çalışması hem de otomatik Swagger dokümantasyonu üretmesi sebebiyle tercih edilmiştir.
- **Veritabanı Katmanı:** Verilerin kalıcı olarak saklanması (persistence) işlemi için geliştirme ve test ortamlarında kurulum kolaylığı sağlayan SQLite veritabanı kullanılmıştır. Veritabanı işlemleri, SQLAlchemy ORM (Object-Relational Mapping) aracı ile soyutlanmıştır.
- **Konteynerleştirme:** Uygulama, Docker kullanılarak konteynerleştirilmiştir. İmaj boyutunu optimize etmek ve güvenlik açıklarını en aza indirmek amacıyla "Multi-stage build" (çok aşamalı inşa) yöntemi kullanılmıştır.



Şekil 1: Sistem Mimari Diyagramı

3. Test Stratejisi

Yazılım test piramidi prensipleri (Test Pyramid) göz önüne alınarak, testlerin büyük çoğunluğunu hızlı çalışan birim (unit) ve entegrasyon testleri oluşturacak şekilde bir strateji izlenmiştir.

- **Test Aracı ve Kapsamı:** Python ekosisteminin endüstri standardı olan `pytest` kullanılmıştır. Proje kapsamında uygulamanın tüm endpoint'lerini kapsayan testler yazılmış ve hedeflenen minimum %70'lik test kapsamı aşılarak **%87 Test Coverage** oranına ulaşılmıştır.
- **Dinamik Veri Üretimi:** Test senaryolarının gerçeği yansıtması için statik veriler ("test1", "test2" vb.) kullanmak yerine, `factory_boy` ve `faker` kütüphaneleri entegre edilmiştir. Bu sayede her test koşulunda rastgele ve benzersiz görev (to-do) verileri üretilerek sistemin farklı girdilere karşı dayanıklılığı ölçülmüştür.

4. Pipeline & Deploy

Sürekli Entegrasyon (CI) süreçlerini otomatize etmek için GitHub Actions kullanılmıştır. Geliştirici tarafından ana depoya kod gönderildiğinde (push işlemi yapıldığında) boru hattı otomatik olarak tetiklenmektedir.

GitHub Actions iş akışı (workflow) sırasıyla şu adımları izler:

1. Kaynak kodun çalışma ortamına çekilmesi (Checkout).
2. Bağımlılıkların (`requirements.txt`) ve test araçlarının kurulması.
3. `flake8` aracı ile PEP8 kodlama standartlarının (Linting) denetlenmesi.
4. `pytest` ile testlerin koşulması ve %70 coverage barajının geçildiğinin doğrulanması.
5. Multi-stage Docker imajının başarıyla inşa (build) edilmesi.

Bu strateji sayesinde, testlerden geçemeyen hiçbir hatalı kodun dağıtım (deploy) aşamasına geçmesine izin verilmemektedir.

5. Performans & Gözlemlenebilirlik

Bu iterasyonda, sistemin temel altyapısının kurulması, test kapsamının artırılması ve Docker tabanlı CI otomasyonunun hatasız çalıştırılmasına öncelik verilmiştir.

Uygulamanın metriklerinin izlenmesi (Prometheus/Grafana) ve yük altındaki performansının ölçülmesi (k6) süreçleri, geliştirme ortamında konsept olarak değerlendirilmiş olup, sistemin üretim (production) ortamına aktarılması sürecinde tam entegrasyonlarının yapılması planlanmıştır. Mevcut mimarinin asenkron FastAPI yapısı sayesinde temel p95 latency (gecikme) değerlerinin düşük kalacağı öngörülmektedir.

6. Sonuç & Öğrendiklerim

Proje sonucunda, %87 test kapsama (coverage) oranına sahip, kod standartları denetlenen ve tek komutla Docker üzerinden çalıştırılabilen tam otomatik bir görev yönetim mikroservisi başarıyla geliştirilmiştir.

- **Karşılaşılan Zorluklar:** Geliştirme sürecinde paket yönetimlerindeki sürüm uyumsuzlukları (özellikle httpx ve FastAPI bağımlılıkları arasındaki çakışmalar) hatalara yol açmış, belirli paket versiyonlarının sabitlenmesiyle (version pinning) bu sorun çözülmüştür.
- **İleride Yapılabilecekler:** Sistemin Kubernetes (Minikube) ortamına dağıtılması (deploy), AWS LocalStack entegrasyonu ile dosya yükleme özelliklerinin eklenmesi ve Grafana üzerinden canlı izleme panellerinin aktif edilmesi projenin bir sonraki aşamasındaki hedeflerdir.

7. Kaynaklar

1. FastAPI Official Documentation:
<https://fastapi.tiangolo.com/>
2. Pytest Documentation:
<https://docs.pytest.org/>
3. Docker Multi-stage Builds:
<https://docs.docker.com/build/building/multi-stage/>
4. GitHub Actions Documentation:
<https://docs.github.com/en/actions>
5. Factory Boy & Faker:
<https://factoryboy.readthedocs.io/>

8. Ekler

Bu bölümde, sistemin konteyner mimarisini ve CI/CD otomasyonunu sağlayan temel yapılandırma dosyalarına yer verilmiştir.

Ek 1: Dockerfile (Multi-stage Build Konfigürasyonu)

Uygulamanın izole bir ortamda paketlenmesini sağlayan ve imaj boyutunu optimize eden Docker dosyasıdır.

Dockerfile

```
# 1. Aşama: Gerekli paketlerin kurulduğu "builder" aşaması
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# 2. Aşama: Uygulamanın çalıştığı ve boyutun küçüldüğü "runner"
aşaması
FROM python:3.11-slim AS runner
WORKDIR /app

# Sadece gerekli olan dosyaları ilk aşamadan kopyalıyoruz
COPY --from=builder /root/.local /root/.local
COPY ./src /app/src

# Python'un kopyalanan paketleri bulabilmesi için yol ayarı
ENV PATH=/root/.local/bin:$PATH

# Uygulamamızı 8000 portunda dışarı açıyoruz
EXPOSE 8000
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port",
"8000"]
```

Ek 2: ci.yml (GitHub Actions İş Akışı)

Koda her müdahale edildiğinde tetiklenen, otomatik test, kod analizi ve docker imaj inşası adımlarını içeren pipeline dosyasıdır.

YAML

```
name: CI/CD Pipeline
```

```
on: [push]
```

```
jobs:
```

```
  build-and-test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Kodu İndir
```

```
        uses: actions/checkout@v4
```

```
      - name: Python Kur
```

```
        uses: actions/setup-python@v5
```

```
        with:
```

```
          python-version: '3.11'
```

```
      - name: Gereksinimleri Yükle
```

```
        run: |
```

```
          pip install -r requirements.txt
```

```
          pip install pytest pytest-cov factory-boy faker httpx
```

```
flake8
```

```
      - name: Lint (Kod Standartları Kontrolü)
```

```
        run: flake8 src/ || true
```

```
      - name: Pytest ile Testleri Çalıştır
```

```
        run: pytest --cov=src --cov-fail-under=70
```

```
      - name: Docker İmajını Oluştur (Build)
```

```
        run: docker build -t myapp:${{ github.sha }} .
```

```
      - name: Deploy ve Smoke Test (Simülasyon)
```

```
        run: |
```

```
          echo "Uygulama başarıyla deploy edildi."
```

```
          echo "Smoke test başarılı: Endpointler yanıt veriyor."
```